DIGITAL FOOTPRINT SCANNER

Scan Modules: Technical Workflow Documentation

How every scanner works under the hood — data sources, request flow, and scoring logic, with deep technical dives into WebRTC IP-leak detection and DNSBL / RBL blacklist checking.

Generated June 16, 2026

Internal engineering reference — not for end-user distribution

Table of Contents

1. Platform Overview **2**

1.1 Two Execution Models 2

1.2 Risk Scoring Convention 2

2. Scan Catalog **3**

Identity & Personal 3

2.1. Email Breach Scanner (Basic + Advanced) 3

2.2. Password Analyzer 4

2.3. Username Mapper 4

2.4. Phone Risk Assessment 4

2.5. CNIC Lookup 5

Digital Presence 5

2.6. Profile Image Search 5

2.7. Image Metadata Detector 6

2.8. Document Analyzer 6

2.9. Ghost Account Finder 7

2.10. Doxxing Vulnerability 7

Network & Device 7

2.11. IP Privacy Scanner 7

2.12. Device Fingerprint 8

2.13. Network Exposure Scanner 8

2.14. Session & Cookie Analyzer 9

2.15. Subdomain Scanner 9

Threats & Awareness 10

2.16. Phishing Quiz 10

2.17. Phishing Vulnerability Profiler 10

2.18. IP Tracker Links 11

Other Modules 11

2.19. Social Media Exposure Simulator 11

2.20. PII Exposure Scan 12

3. Deep Dive — WebRTC IP Leak Detection **13**

3.1 Why This Is Possible: ICE and STUN 13

3.2 Exact Implementation in This App 13

3.3 Interpreting the Result 14

3.4 Mitigation Recommended In-App 14

4. Deep Dive — DNSBL / RBL Blacklist Scanning **15**

4.1 Why DNS, Specifically? 15

4.2 Exact Implementation in This App 15

4.3 Why an IP Ends Up Listed 16

1. Platform Overview

The Digital Footprint Scanner is a web application that lets a user check how exposed their personal information is across breach databases, public records, social platforms, and their own browser/network configuration. It is split into a React/TypeScript single-page frontend and a Node.js/Express backend. Some scans run entirely in the browser (no backend involved, e.g. Device Fingerprint, Session & Cookie Analyzer, Password Analyzer, Phishing Quiz); others call the Express backend, which in turn queries third-party OSINT and breach-intelligence APIs (LeakCheck.io, SerpAPI, HackerTarget, crt.sh, ip-api.com, proxycheck.io, simowner.net.pk) or performs raw network-level checks (DNS blacklist lookups).

This document explains, scan by scan, exactly what happens between the moment a user clicks “Scan” and the moment results render — which APIs are hit, what is computed locally, and how the final risk score or verdict is derived. Two mechanisms get an extended technical deep dive because they are protocol-level checks rather than simple API lookups: WebRTC IP-leak detection (Section 3) and DNSBL/RBL blacklist scanning (Section 4).

1.1 Two Execution Models

- Client-side only: JavaScript running in the user's own browser inspects browser/network APIs directly (WebRTC, cookies, localStorage, navigator.*, screen/canvas properties, response headers). Nothing is sent to the backend except, optionally, the final result for the user's saved scan history.
- Backend-proxied: the frontend POSTs/GETs to an Express route (e.g. /scan-ip, /email-scan, /scan-doxxing). The backend calls one or more external APIs server-side — partly because some APIs require a secret key that must never reach the browser, and partly to avoid CORS restrictions and to enrich/normalize results before sending them back.

1.2 Risk Scoring Convention

Most scans produce a 0–100 risk score and a severity band. While each scan's scoring formula is bespoke (detailed per-scan below), the bands are used consistently across the app:

Low / Safe	Little to no exposure found; informational only.
Moderate / Warning	Some exposure exists but is low-sensitivity (e.g. username reuse, public profile photo).
High	Sensitive data exposed (phone, address, DOB, password hash) or a real leak path found (WebRTC/DNS leak).
Critical	Plaintext credentials, financial data, or government ID exposed — immediate action recommended.

2. Scan Catalog

Each entry below follows the same structure: what the scan tells the user, exactly which data sources/APIs back it, the step-by-step request flow, and how the result is scored or interpreted.

Identity & Personal

2.1. Email Breach Scanner (Basic + Advanced)

Category: Identity & Personal

Data Sources / APIs

- LeakCheck.io free public API (https://leakcheck.io/api/public?check=<email>) — always used for the Basic scan, no key required.
- LeakCheck.io paid v2 API (https://leakcheck.io/api/v2/query/<email>, header X-API-Key) — used only for the Advanced scan.
- Internal BREACH_META lookup table — maps known breach names (LinkedIn, Adobe, Yahoo, RockYou, etc.) to domain + record-count metadata for display.

Step-by-Step Workflow

1. User enters an email and clicks “Basic Scan” (always enabled when the email is valid). The frontend POSTs { email, mode: "basic" } to /email-scan.
2. Backend calls the LeakCheck free API through a shared leakcheckGet() helper (an axios instance bound to an IPv4-only HTTPS agent — see Section 5.1 for why).
3. Backend cross-references each returned breach/source name against BREACH_META to attach a domain and an approximate record count, and computes a severity per breach via getBreachSeverity() (password/SSN/CC → critical, hash/DOB → high, phon else low).
4. Results render as breach cards. If at least one breach was found (hasLeaks = true), the “Advanced Scan” button unlocks.
5. If the user clicks Advanced Scan, the frontend POSTs { email, mode: "advanced" } to /email-scan. The backend requires LEAKCHECK_API_KEY to be configured; if missing it never attempts the call.
6. The paid v2 endpoint returns per-record actual field values (not just field names) — email, password, password_hash/hash/md5/sha1, username, name, phone, address, ip, dob, line, grouped under a source object with name/breach_date/unverified.
7. Every field returned by the paid API is rendered (none are silently dropped), with sensitive fields (password, password_hash, hash, md5, sha1, sha256, bcrypt, line) blurred by default behind a per-field eye-icon reveal toggle.
8. If the paid call fails (quota exceeded or invalid key), the backend returns HTTP 429 with LeakCheck's own error message and the UI surfaces it as an inline error — it does not silently fall back, since the user explicitly chose to spend paid quota.

Output & Interpretation

Basic scan output is a free-tier breach list (which breaches, approximate dates, exposed field types) used to decide whether spending paid quota is worth it. Advanced scan output is the actual leaked values from each breach record, which is far more sensitive and is gated behind the Basic scan finding something first — this two-tier design exists specifically so clean emails never consume paid API quota.

2.2. Password Analyzer

Category: Identity & Personal

Data Sources / APIs

- Have I Been Pwned (HIBP) Pwned Passwords API — https://api.pwnedpasswords.com/range/<5-char SHA-1 prefix>, called directly from the browser (no backend involved).
- A local list of ~140 common/weak passwords and a leetspeak-normalization table, both bundled client-side.

Step-by-Step Workflow

1. User types a password. SHA-1 is computed entirely client-side via the browser's crypto.subtle.digest API — the plaintext password never leaves the browser.
2. Only the first 5 hex characters of the hash (the “prefix”) are sent to HIBP's range endpoint; HIBP returns every suffix+count pair sharing that prefix (k-anonymity model — HIBP never sees the full hash or the password).
3. The frontend searches the returned suffix list for its own suffix locally to learn the exact breach-appearance count, if any.
4. In parallel, the password is checked against the common-password set (after de-leeting substitutions like @!’a, 0!’o, 3!’e) and scored for entropy: length, character-class diver patterns.

Output & Interpretation

Combines two signals: (a) “seen in N known breaches” from HIBP (any count > 0 means the password is unsafe regardless of its complexity), and (b) a structural strength score for passwords with zero breach hits. A long random password that still appears in HIBP is flagged just as critically as a short common one.

2.3. Username Mapper

Category: Identity & Personal

Data Sources / APIs

- GitHub Users API, Reddit (/user/<u>/about.json), GitLab Users API, Hacker News Firebase API, Dev.to Users API — all called directly client-side, unauthenticated, one HTTP request per platform.

Step-by-Step Workflow

1. User enters a username. The frontend fires one fetch() per supported platform in parallel.
2. Each platform's response (HTTP 200 + valid JSON vs 404) determines whether the account exists.
3. Where the API exposes it, profile metadata (account age, karma/reputation, bio) is parsed and displayed alongside the existence flag.

Output & Interpretation

A per-platform existence matrix showing exactly where the username is registered, useful for understanding cross-platform identity correlation risk (the more platforms reuse one handle, the easier an attacker can link separate accounts to one real person).

2.4. Phone Risk Assessment

Category: Identity & Personal

Data Sources / APIs

- Internal carrier-prefix and area-code tables (PK_MOBILE_PREFIXES, PK_LANDLINE_AREAS, COUNTRY_META) — fully local, no external call for the basic validity/carrier check.

- simowner.net.pk (scraped via /scan-phone, Pakistan numbers only) — a public SIM-registration lookup site queried server-side with spoofed Pakistani-residential request headers (the hosting provider's datacenter IP is otherwise blocked by the target site).

Step-by-Step Workflow

1. User enters a phone number and country. GET /scan-phone?phone=...&country=... is called.
2. Backend normalizes the number (strips country code/leading zero) and classifies it as Mobile or Landline using the prefix/area-code tables, deriving carrier (Jazz/Zong/Ufone/Telenor/PTCL) and city/province for Pakistani numbers.
3. For Pakistani mobile numbers, the backend additionally queries simowner.net.pk's ajax-handler.php endpoint, presenting a forged X-Forwarded-For / Referer / User-Agent header set so the request appears to originate from inside Pakistan.
4. If a registration record is returned, the registered owner name, masked/formatted CNIC, and address are merged into the response (this part of the UI was later removed per product decision — the backend still receives the data, but the frontend no longer displays the simowner-sourced owner card).

Output & Interpretation

Tells the user what any caller/SMS-blocker app would already infer (carrier, region, number validity) plus, where available, whether their number is tied to a public registration record exposing personal identity.

2.5. CNIC Lookup

Category: Identity & Personal

Data Sources / APIs

- simowner.net.pk ajax-handler.php, queried by CNIC number instead of phone number.

Step-by-Step Workflow

1. User enters a 13-digit Pakistani CNIC. GET /scan-cnic?cnic=... is called; the backend rejects anything that isn't exactly 13 digits.
2. Backend queries simowner.net.pk with the same spoofed-header technique used by the phone scan.
3. The response (if any) is a list of every SIM registered against that CNIC: owner name, mobile number (carrier inferred from prefix), and address.

Output & Interpretation

Shows the user how many phone numbers/SIMs are publicly traceable to their national ID, illustrating identity-to-telecom linkage exposure.

> **NOTE**
> This data source is a third-party public lookup site, not an official NADRA/PTA database, and its own data note states coverage may be incomplete or dated — results should be treated as indicative, not authoritative.

Digital Presence

2.6. Profile Image Search

Category: Digital Presence

Data Sources / APIs

- exifr (npm) — local EXIF parsing of the uploaded image.

- imgbb API — temporary public image hosting so the photo has a URL SerpAPI can crawl.
- SerpAPI Google Lens engine — reverse image search against the hosted URL.

Step-by-Step Workflow

1. User uploads an image (multipart/form-data) to POST /scan-image.
2. Backend first runs exifr.parse() locally to pull camera make/model, capture date, software, GPS coordinates, and dimensions — this never leaves the server.
3. The image is streamed (not base64-inlined) to imgbb to obtain a temporary public URL.
4. That URL is handed to SerpAPI's Google Lens engine, which returns visually-similar/matching results across the indexed web.

Output & Interpretation

Surfaces every place online a reverse-image-search engine can find a visual match for the photo (other social profiles, reposts, stock-photo theft, etc.), plus any embedded EXIF metadata the original file leaked.

2.7. Image Metadata Detector

Category: Digital Presence

Data Sources / APIs

- exifr (npm) — full EXIF/GPS/XMP/IPTC parser, used locally on the backend, no external API.

Step-by-Step Workflow

1. User uploads an image to POST /scan-metadata.
2. Backend runs exifr.parse() with tiff/exif/gps/xmp/iptc extraction enabled.
3. GPS latitude/longitude (if present) is rounded to 6 decimal places and used to build a ready-to-click Google Maps URL; camera make/model/serial, lens, capture/modify timestamps, exposure settings (ISO, shutter, aperture, focal length) are all extracted.

Output & Interpretation

Demonstrates exactly what a stranger could learn from a photo's hidden metadata alone — most critically, the exact GPS coordinates where it was taken, which is invisible in the rendered image but readable by any EXIF-aware tool.

2.8. Document Analyzer

Category: Digital Presence

Data Sources / APIs

- Local file-format detection (magic-byte signatures), exifr, and custom PDF/OOXML metadata + threat-heuristic parsers — entirely local, no external API.

Step-by-Step Workflow

1. User uploads a document to POST /analyze-document.
2. Backend computes a SHA-256 hash of the raw bytes and detects the true file type from its magic bytes (detectMagicType), independent of the file extension or declared MIME type.
3. If the declared extension doesn't match the detected magic type (e.g. a .jpg that's actually a PDF), the response flags a magicMismatch — a common technique used to smuggle malicious files past naive extension filters.
4. Format-specific metadata extraction runs: extractPdfMeta() + analyzePdfThreats() for PDFs (looking for embedded JavaScript, auto-launch actions, suspicious objects); extractZipDocMeta() for OOXML files (docx/xlsx/pptx, which are ZIP containers); exifr for embedded images.

Output & Interpretation

Reveals hidden author names, editing software, and creation timestamps embedded in office documents and PDFs (often left over from earlier drafts or from a corporate template), plus flags structurally suspicious or disguised files.

2.9. Ghost Account Finder

Category: Digital Presence

Data Sources / APIs

- A fixed list of platforms (GHOST_PLATFORMS) probed for account existence.
- SerpAPI Google search — general web mentions plus a targeted site:pastebin.com OR site:ghostbin.com OR site:hastebin.com query.
- LeakCheck.io free public API, if an email is supplied.

Step-by-Step Workflow

1. User supplies a username and/or email. POST /scan-ghost runs three checks in parallel: per-platform existence probes, two-to-three SerpAPI searches, and (if email given) a LeakCheck free lookup.
2. Platform probe results, organic search mentions, and paste-site mentions are merged and deduplicated by domain.

Output & Interpretation

Designed to surface forgotten/abandoned (“ghost”) accounts the user may not remember creating, plus any pastebin-style leak that name-drops the handle — a common source of old, never-rotated credentials.

2.10. Doxxing Vulnerability

Category: Digital Presence

Data Sources / APIs

- A fixed list of platforms (DOXXING_PLATFORMS) probed for account existence.
- SerpAPI Google search — general mentions of the username/email plus a targeted site:pastebin.com query.

Step-by-Step Workflow

1. User supplies a username and/or email. POST /scan-doxxing runs platform existence checks and SerpAPI searches in parallel (mirroring the Ghost Account Finder, but framed around public-exposure risk rather than forgotten accounts).
2. Results are split into organic web mentions vs. pastebin mentions specifically, since pastebin hits are the strongest doxxing-risk signal.

Output & Interpretation

Estimates how easily someone could compile a dossier on the user just from public search results tied to one handle or email.

Network & Device

2.11. IP Privacy Scanner

Category: Network & Device

Data Sources / APIs

- ip-api.com — free geolocation + a broad proxy/hosting boolean flag.

- proxycheck.io — free VPN/Tor/proxy type classification.
- DNSBL/RBL checks against 35 public blacklists (see Section 4 for the full mechanism).
- RTCPeerConnection (browser-side, see Section 3) — only runs when the IP being scanned is the user's own current IP.
- A local VPN_ISP_KEYWORDS list (NordVPN, ExpressVPN, etc.) used as a last-resort classifier when the paid-style signals are inconclusive.

Step-by-Step Workflow

1. User enters an IP (or the page auto-fills the visitor's own IP via api.ipify.org). The frontend POSTs to /scan-ip.
2. Backend fires three checks in parallel with Promise.allSettled: ip-api.com geolocation, proxycheck.io classification, and (for IPv4) the DNSBL sweep.
3. VPN/proxy/Tor classification is layered: proxycheck.io's explicit type is trusted first; ip-api.com's broader proxy/hosting flags are used as a fallback signal; ISP name keyword-matching is the last resort.
4. If the IP under scan is the user's own browser session, the frontend additionally runs the client-side WebRTC leak test and merges it into the displayed results (a remote/arbitrary IP cannot be WebRTC-tested, since that test only ever reveals the testing browser's own network path).

Output & Interpretation

A composite privacy verdict: geolocation, ISP/ASN, VPN/Proxy/Tor/hosting flags with provider name where known, DNSBL blacklist hit count, and (for self-scans) a live WebRTC leak result.

2.12. Device Fingerprint

Category: Network & Device

Data Sources / APIs

- Fully client-side: navigator.*, screen.*, Intl.DateTimeFormat, and an HTML5 canvas rendering fingerprint — no backend or external API.

Step-by-Step Workflow

1. Browser/OS/screen/timezone/language/hardware properties are read directly from the DOM and Navigator APIs.
2. A canvas fingerprint is generated by drawing fixed text/shapes to an offscreen <canvas> and hashing the resulting pixel data — subtle differences in GPU, font rendering, and anti-aliasing make this hash highly device-specific.
3. An entropy/uniqueness score is built additively from how unusual each individual signal is (e.g. non-standard screen resolution, very high core count, large language list, high device-pixel-ratio all push the score up).

Output & Interpretation

Estimates how trackable the browser is via fingerprinting alone, i.e. without cookies — a higher uniqueness score means the device profile is rare enough to re-identify the user across sites even after clearing cookies.

2.13. Network Exposure Scanner

Category: Network & Device

Data Sources / APIs

- Backend /network-info (server-observed IP + geolocation + raw request headers).

- RTCPeerConnection (WebRTC leak test — Section 3).
- Cloudflare DNS-over-HTTPS (https://cloudflare-dns.com/dns-query) for DNS resolver-leak detection.
- Local browser fingerprint + Navigator.connection (effective type/downlink/RTT) + HTTPS-vs-HTTP check.

Step-by-Step Workflow

1. On scan start, geo info and exposed HTTP headers are fetched from the backend's /network-info endpoint (server-side, since headers like User-Agent are only visible to whoever receives the actual HTTP request); if that fails, ipapi.co is used as a client-side fallback.
2. The WebRTC leak test runs (detectWebRTC(), detailed in Section 3).
3. A DNS-leak test runs: the browser queries Cloudflare's DoH resolver for a TXT record on whoami.cloudflare, which echoes back the IP of whichever resolver actually serviced the query. If that IP differs from the user's known public IP, their traffic is being resolved by an unexpected DNS server (e.g. their ISP's resolver leaking outside a VPN tunnel).
4. Browser fingerprint, response headers, and connection quality (navigator.connection) are gathered locally.
5. computeRisk() combines all six checks into one weighted score: unencrypted HTTP (+30), WebRTC leak (+25 to +35), DNS leak (+15), plus smaller fingerprint/header-based additions.

Output & Interpretation

The most comprehensive single-page exposure report in the app — 7 checks (IP/geo, WebRTC, DNS leak, fingerprint, HTTPS, headers, connection) rolled into one risk score with a category-by-category breakdown and concrete remediation steps per failed check.

2.14. Session & Cookie Analyzer

Category: Network & Device

Data Sources / APIs

- Fully client-side: document.cookie, localStorage, sessionStorage, and Navigator properties — no backend or external API.

Step-by-Step Workflow

1. All cookies are parsed from document.cookie and pattern-matched for likely auth/session tokens (looksLikeAuth, looksLikeSession) and for JWT structure (three base64url segments separated by dots) in both cookies and Web Storage.
2. localStorage and sessionStorage are enumerated for any token-shaped values.
3. The current URL is checked for tokens leaked into query parameters (a common mistake that exposes session tokens in browser history, server logs, and the Referer header).
4. A browser security profile is captured: HTTPS vs HTTP, WebRTC availability, Service Worker support, and whether third-party cookies are likely blocked (inferred from the presence of the modern cookieStore API).

Output & Interpretation

Flags risky client-side storage patterns: auth tokens stored in plain localStorage (readable by any XSS payload, unlike httpOnly cookies), tokens embedded in URLs, and missing HTTPS — issues a real attacker's injected script (illustrated in-app with a literal XSS-via-cookie-theft example) could exploit.

2.15. Subdomain Scanner

Category: Network & Device

Data Sources / APIs

- HackerTarget hostsearch API (https://api.hackertarget.com/hostsearch/?q=<domain>) — returns subdomain,IP pairs from its own DNS index.
- crt.sh Certificate Transparency log search (https://crt.sh/?q=%.<domain>&output=json) — used as a fallback/supplement source.

Step-by-Step Workflow

1. User enters a domain. GET /scan-subdomains?domain=... is called; the input is normalized (protocol/path/www. stripped).
2. Backend queries HackerTarget first, parsing the CSV-style subdomain,IP response.
3. Backend additionally queries crt.sh, which returns every publicly-issued TLS certificate covering that domain or any subdomain (because Certificate Transparency logs are public and append-only, this is one of the most complete subdomain-discovery techniques available without active DNS brute-forcing).
4. Results from both sources are merged and de-duplicated; subdomains matching an INTERESTING regex (admin, api, staging, vpn, backup, jenkins, grafana, etc.) are flagged as higher-value targets for an attacker.

Output & Interpretation

A list of discovered subdomains with their resolved IPs where known, highlighting administrative/internal-sounding hosts that are often forgotten and under-secured.

Threats & Awareness

2.16. Phishing Quiz

Category: Threats & Awareness

Data Sources / APIs

- Fully client-side: a static bank of example phishing/legitimate message scenarios bundled in the app.

Step-by-Step Workflow

1. The user is shown a series of realistic email/SMS scenarios and asked to classify each as phishing or legitimate.
2. Answers are scored locally against the known-correct classification for each scenario.

Output & Interpretation

An awareness/education score, not a live data-exposure scan — it measures the user's own ability to recognize phishing patterns (urgency cues, lookalike domains, mismatched sender addresses, etc.).

2.17. Phishing Vulnerability Profiler

Category: Threats & Awareness

Data Sources / APIs

- SerpAPI Google search, if SERPAPI_KEY is configured server-side; otherwise the backend falls back to a deterministic simulation based purely on the fields the user supplied.

Step-by-Step Workflow

1. User supplies name, and optionally email/LinkedIn URL/location. POST /scan-phishing-profile is called.
2. If a SerpAPI key is configured, the backend searches for public mentions of the name (and email, if

given) to estimate how much OSINT material a real attacker could gather to craft a convincing spear-phishing pretext.
3. If no key is configured, the backend instead returns a risk score built additively from which fields were supplied (email present, location present, LinkedIn present each add points) plus a generated example spear-phishing scenario illustrating how that specific combination of public info could be weaponized.

Output & Interpretation

Models an attacker's reconnaissance phase: the more public information tied to a name (employer, location, social profiles), the more convincing a targeted phishing pretext can be made — the report includes a worked example message.

2.18. IP Tracker Links

Category: Threats & Awareness

Data Sources / APIs

- ip-api.com (geolocation of clicks) plus a Supabase-backed trackers / tracker_hits table for storage.

Step-by-Step Workflow

1. An authenticated user creates a tracking link (POST /tracker/create), which generates a short token mapped to a redirect URL the user controls.
2. When anyone clicks the generated /t/<token> link, the backend looks up the tracker, captures the visitor's IP (from X-Forwarded-For/socket address), User-Agent (parsed into browser/OS/device), and Referer.
3. The visitor's IP is geolocated via ip-api.com (country/city/ISP/lat-lon) and the hit is inserted into tracker_hits before an interstitial “Loading...” page silently redirects the visitor on to the real destination URL.
4. The link owner views aggregated hits (GET /tracker/:token/hits) to see who clicked, from where, and when.

Output & Interpretation

This is the one scan in the catalog that is offensive/educational by design rather than defensive: it demonstrates, first-hand, exactly how trivially a shared link can capture a clicking visitor's IP/location/device — the same technique used in real phishing and OSINT operations — framed here as an awareness tool for the link owner, not as a tool to target third parties without consent.

Other Modules

2.19. Social Media Exposure Simulator

Category: Digital Presence

Data Sources / APIs

- None external — this module does not call any live social-media API.

Step-by-Step Workflow

1. User selects a platform (Twitter/X, Facebook, LinkedIn, TikTok), enters a username, and indicates whether the account is private.
2. A deterministic pseudo-random score is derived from a simple hash of (username + platform) rather than a real lookup — private accounts on platforms where that matters (TikTok, Facebook) are scored noticeably better than public ones, since posts/photos genuinely are hidden from strangers on

those platforms.
3. A fixed checklist of typical profile-exposure settings (full name visible, profile photo public, bio public, etc.) is rendered with exposure flags derived from the same hash.

Output & Interpretation

Functions as an illustrative privacy-settings checklist rather than a live scan of the user's actual account — it is useful for walking through what “public vs private” typically exposes on each platform, but the specific findings shown are simulated, not fetched from the real platform.

> **NOTE**
> Unlike every other module in this catalog, this scan does not query a live external source. Treat its output as educational/illustrative, not as a factual report of the named account's real privacy posture.

2.20. PII Exposure Scan

Category: Identity & Personal

Data Sources / APIs

- LeakCheck.io free public API (only for the email field). All other fields — phone, name, date of birth, SSN last-4, address, username — are validated structurally rather than looked up against a live database.

Step-by-Step Workflow

1. User supplies any subset of: email, phone, full name, date of birth, SSN last-4, address, username. POST /scan-pii is called with whichever fields were filled in.
2. If an email was supplied, the backend performs a real LeakCheck free-tier lookup and uses the breach count + exposed field types to set that field's severity (critical if credentials or PII were exposed in a breach, otherwise scaled by breach count).
3. Every other supplied field is scored heuristically based on category-level real-world exposure norms baked into the backend (e.g. phone numbers are flagged high-risk because they appear in virtually all major data-broker databases; SSNs are treated as inherently critical if present at all) — these are not individually verified against a live database the way email is.

Output & Interpretation

A field-by-field exposure report combining one real breach-database check (email) with general data-broker exposure norms for everything else, giving a holistic sense of identity-theft surface area across multiple PII types at once.

> **NOTE**
> Only the email field reflects a real, queried result. Phone/name/DOB/SSN/address/username severities are heuristic category risk, not confirmation that those specific values were found exposed anywhere.

3. Deep Dive — WebRTC IP Leak Detection

WebRTC (Web Real-Time Communication) is a browser standard for direct peer-to-peer audio/video/data connections — the technology behind in-browser video calls. To make a direct peer-to-peer connection possible, a browser must first discover every network path it could be reached on, including paths that exist purely for local-network or NAT-traversal purposes and were never meant to be exposed to a website. This discovery process is exactly what this scanner uses (in the same way a real attacker or tracking script would) to detect whether a user's real IP address is leaking outside their VPN tunnel.

3.1 Why This Is Possible: ICE and STUN

WebRTC connection setup uses a framework called ICE (Interactive Connectivity Establishment). Before two peers can talk directly, each browser gathers a list of “candidate” addresses it could be reached on:

- Host candidates — the browser's own local network interface IP(s), e.g. a private 192.168.x.x or 10.x.x.x address assigned by the router.
- Server-reflexive candidates — obtained by asking a public STUN server “what IP and port do you see me connecting from?”. STUN (Session Traversal Utilities for NAT) servers exist purely to answer this question; this app uses Google's public STUN server (stun:stun.l.google.com:19302).
- Relay candidates — obtained from a TURN server when direct connection isn't possible at all (not used by this scanner; only STUN is configured).

Crucially, this address-gathering process happens at the browser/OS networking layer, below the VPN's application-layer routing in many VPN configurations. A VPN reroutes general internet traffic, but a browser asking “what are all my local network interfaces, and what does an external STUN server see me as?” can, depending on the VPN client and OS, still report the real underlying network interface IP and/or reach the STUN server outside the VPN tunnel entirely. The result: a website can learn the user's real IP even while their visible browsing IP (as seen by, say, a normal HTTP request) shows the VPN's IP.

3.2 Exact Implementation in This App

The detection logic lives entirely client-side in detectWebRTC() (NetworkScan.tsx) and is reused by the IP Privacy Scanner when the IP under test is the user's own:

```
const pc = new RTCPeerConnection({
  iceServers: [{ urls: "stun:stun.l.google.com:19302" }]
});
pc.createDataChannel("");          // forces ICE gathering to start
pc.onicecandidate = (e) => {
  if (!e.candidate) { /* gathering finished */ resolve(res); return; }
  const ips = e.candidate.candidate.match(/(\d+\.\d+\.\d+\.\d+)/g) ?? [];
  ips.forEach(ip => {
    if (isPrivateRange(ip)) localIPs.push(ip);   // 10.x / 192.168.x / 172.16-31.x
    else if (isRoutablePublicIP(ip)) publicIPs.push(ip);
  });
};
pc.createOffer().then(o => pc.setLocalDescription(o));
setTimeout(() => { pc.close(); resolve(res); }, 5000); // 5s safety timeout
```

1. An RTCPeerConnection is created with one public STUN server configured — no actual call or peer is ever connected; the connection object exists purely to trigger ICE candidate gathering.
2. A dummy data channel is created. This is required because Chrome/Firefox only start gathering ICE

candidates once there is at least one media line or data channel to negotiate.

3. pc.createOffer() + setLocalDescription() formally kicks off SDP (Session Description Protocol) negotiation, which is what actually triggers the browser to enumerate host candidates and query the STUN server for a server-reflexive candidate.
4. Every candidate the browser discovers fires the onicecandidate event. Each candidate string (an SDP “a=candidate” line) is regex-matched for IPv4 addresses.
5. Each extracted IP is classified: addresses in the RFC 1918 private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) are local/host candidates — these are the user's real LAN IP, which should never be visible to a remote website under normal circumstances. Anything else routable (not 0.0.0.0, not 127.x loopback) is treated as a leaked public IP — this is the STUN server-reflexive candidate, i.e. the real public IP the browser is using on the wire, independent of whatever IP a normal HTTP request to the site would show.
6. Gathering ends either when the browser fires a null candidate (signaling “gathering complete”) or after a 5-second timeout, whichever comes first.
7. If candidate creation itself throws (e.g. RTCPeerConnection is unavailable or blocked by a browser extension/setting), the result is marked blocked rather than leaked — this is the desired, safe state.

3.3 Interpreting the Result

LEAKED
Any local or public IP was discovered via ICE candidates. If the user is browsing through a VPN and a public IP different from the VPN's exit IP appears here, that is the user's real ISP-assigned IP leaking outside the tunnel — the single most damaging finding this app can produce, since it defeats the entire purpose of using a VPN for anonymity.

SAFE (no leak)
ICE gathering completed but produced no usable candidates worth flagging, or completed without exposing anything beyond what's already visible from a normal connection. This is the common, expected outcome for browsers with built-in WebRTC IP-leak protections (e.g. Chrome's “use local IP only” mDNS mode) or with VPN clients that correctly tunnel STUN traffic too.

BLOCKED
RTCPeerConnection threw an error or never produced any candidates — typically because WebRTC is disabled in browser settings (e.g. media.peerconnection.enabled=false in Firefox about:config) or a privacy extension (uBlock Origin's “Prevent WebRTC leaks,” or a dedicated WebRTC-blocking add-on) intercepted it. This is the most secure state: WebRTC simply cannot be used to leak anything because it cannot run at all.

3.4 Mitigation Recommended In-App

- Enable a WebRTC-leak-blocking option in an ad/privacy-blocking extension (the app specifically suggests uBlock Origin's built-in setting).
- On Firefox, disable WebRTC entirely via about:config !’ media.peerconne
- Use a VPN client/router configuration known to correctly route UDP STUN traffic through the tunnel, not just TCP web traffic.

4. Deep Dive — DNSBL / RBL Blacklist Scanning

A DNSBL (DNS-based Blackhole List), also called an RBL (Realtime Blackhole List), is a publicly queryable list of IP addresses known or suspected to be sources of spam, malware, botnet traffic, or other network abuse. DNSBLs are one of the oldest and most widely deployed anti-abuse mechanisms on the internet — mail servers worldwide consult them before accepting an incoming connection, and many security tools (including this scanner) consult them to flag an IP as having a history of abusive behavior.

4.1 Why DNS, Specifically?

DNSBLs reuse the existing DNS protocol as their query mechanism instead of requiring a custom protocol, because DNS is universally available, heavily cached, and extremely fast. The trick is in how the question is encoded as a domain name: to ask “is IP address 1.2.3.4 listed on blacklist example.org?”, the IP's octets are reversed and prepended as subdomain labels in front of the blacklist's own zone name:

```
IP to check:        1.2.3.4
Reversed octets:     4.3.2.1
DNSBL zone:           dnsbl-1.uceprotect.net
Query made:           4.3.2.1.dnsbl-1.uceprotect.net   (type A)
```

If that IP is on the blacklist, the DNSBL's authoritative nameserver responds with an A record (almost always something in the 127.0.0.x range, used as a status/return code rather than a real routable address). If the IP is clean, the nameserver returns NXDOMAIN (“no such domain”) — there is no record to find, because no listing for that reversed-IP subdomain exists in the zone. This NXDOMAIN-means-clean behavior is the whole basis of the check: a successful DNS resolution is the bad outcome, and a failed lookup is the good one.

4.2 Exact Implementation in This App

The backend implements this directly with Node's built-in dns module — no external library or paid API is used:

```
function reverseIPv4(ip) {
  return ip.split(".").reverse().join(".");
}

async function checkRBL(ip) {
  const reversedIP = reverseIPv4(ip);
  let listed = [];
  for (const rbl of RBL_SERVERS) {            // 35 servers, sequential
    const lookup = `${reversedIP}.${rbl}`;
    try {
      await dns.resolve4(lookup);              // resolves -> LISTED
      listed.push(rbl);
    } catch (err) { /* NXDOMAIN -> not listed, ignore */ }
  }
  return { totalChecks: RBL_SERVERS.length, listedCount: listed.length, listedOn: listed
};
}
```

1. The target IP's octets are reversed (1.2.3.4 ⁉ 4.3.2.1) per the DNSBL co

2. For each of 35 configured DNSBL zones (RBL_SERVERS — including well-established lists like Spamhaus's xbl/sbl, SpamCop, Barracuda Central, UCEPROTECT levels 1-3, DroneBL, and several smaller abuse-focused lists), the backend builds <reversed-ip>.<zone> and performs an A-record lookup with dns.resolve4().
3. A successful resolution means that zone considers the IP listed; the zone's hostname is recorded in listed[].
4. A rejected promise (DNS error, almost always NXDOMAIN) is caught and silently treated as “not listed on this zone” — this is expected, normal behavior for the vast majority of IPs on the vast majority of zones, not a fault condition.
5. After all 35 zones have been checked, the function returns the total number of zones checked, how many returned a positive listing, and the specific zone hostnames that flagged the IP.

This whole check only runs for IPv4 addresses (DNSBL infrastructure is overwhelmingly IPv4-only in practice) and is fired in parallel with the ip-api.com geolocation call and the proxycheck.io VPN-classification call inside POST /scan-ip, using Promise.allSettled so a slow or failing DNSBL sweep can never block the rest of the IP report from rendering.

4.3 Why an IP Ends Up Listed

- It has previously sent spam (open relay, compromised mail server, or a spambot running on a compromised machine on that network).
- It is a known Tor exit node, open proxy, or VPN egress IP that has been abused by other users sharing the same exit address — this is extremely common for shared/cheap VPN and hosting IPs, and is one reason this scanner cross-references DNSBL results alongside the separate VPN/proxy/hosting classification rather than treating them as unrelated signals.
- It belongs to a dynamic residential IP range (DUL-style listings, e.g. dyna.spamrats.com) that ISPs assign to home connections — listed preemptively because residential IPs should rarely run their own outbound mail server, so several lists block the whole range as a category, not because that specific IP did anything wrong.
- It is part of a netblock previously compromised in a botnet (e.g. flagged by DroneBL, which specifically tracks botnet/drone activity).

4.4 Interpreting the Result

listedCount = 0
Clean across all 35 zones queried — the IP has no known history of abuse reported to any of these public blacklists. This is the expected, normal result for the overwhelming majority of legitimate residential and business IPs.

listedCount > 0
The specific zone names in listedOn matter: a single hit on a DUL/dynamic-range list (e.g. dyna.spamrats.com) for a home internet connection is low-signal and often unrelated to the current user's behavior. Multiple hits, or any hit on a reputation-focused list like Spamhaus's sbl/xbl or Barracuda Central, is a much stronger signal that the IP (or a previous user of that same IP, in the case of shared hosting/VPN/Tor exit nodes) has an actual abuse history.

4.5 Practical Limitations

- DNSBL lookups reflect the IP's history, not the current user's behavior — this matters enormously for shared, dynamic, or VPN/hosting IPs, where the person being flagged may be entirely unrelated to

whoever caused the original listing.

- Lists update independently and can lag real-world changes by hours to weeks in either direction (an IP can stay listed well after the abusive traffic stopped, or take time to get listed after abuse begins).
- The check runs synchronously, zone by zone, inside a single request — with 35 zones it is the slowest single component of the IP Privacy Scanner, which is part of why it's parallelized with (not blocking) the geolocation and VPN-classification calls.

5. Architecture Notes

5.1 The IPv4-Forcing Fix for LeakCheck.io

Worth documenting because it's a real example of an environment-specific networking bug: under WSL2, IPv6 routing to Cloudflare-fronted hosts (including leakcheck.io) is broken, causing both Node's built-in fetch() (undici) and plain axios.get() calls to randomly attempt an IPv6 address first, hang, and fail with ETIMEDOUT/ENETUNREACH even though IPv4 connectivity to the same host worked fine. The fix was a shared, IPv4-pinned HTTPS agent reused by every leakcheck.io call site in the backend:

```
const leakcheckAgent = new https.Agent({ family: 4, keepAlive: false });
const leakcheckGet = (url, opts = {}) =>
  axios.get(url, { ...opts, httpsAgent: leakcheckAgent, timeout: opts.timeout || 15000
});
```

All four leakcheck.io call sites in server.js (email scan, doxxing scan, ghost scan, PII scan) route through this single helper, so the fix and the timeout behavior are centralized rather than duplicated.

5.2 Why Some Scans Never Touch the Backend

Device Fingerprint, Session & Cookie Analyzer, Password Analyzer (beyond the HIBP call), Phishing Quiz, and the WebRTC/DNS-leak portions of the Network Exposure Scanner are deliberately implemented client-side only. Two reasons drive this: first, several of these checks are only meaningful when run from the user's own browser (cookies, localStorage, canvas fingerprint, and WebRTC candidates are all properties of the specific browser instance running the page — a server has no access to them and could not replicate the check). Second, for password checking specifically, keeping the SHA-1 computation client-side and only ever transmitting a 5-character hash prefix to HIBP is a deliberate, load-bearing privacy property (k-anonymity) that would be defeated if the password were sent to this app's own backend at all.

6. Disclaimer & Responsible Use

This documentation describes scan mechanisms intended for checking exposure of one's own data, or data the operator is explicitly authorized to assess. Several modules (Doxxing Vulnerability, Ghost Account Finder, Phishing Vulnerability Profiler, and especially IP Tracker Links) are built around techniques that are equally usable for legitimate self-assessment and for targeting a third party without consent. The IP Tracker module in particular logs IP, approximate location, and device information for anyone who clicks a generated link — it must only be used on links shared with people who are aware they may be tracked (e.g. for a security-awareness demo), not distributed covertly to capture an unsuspecting third party's location.

Third-party data sources referenced throughout this document (LeakCheck.io, simowner.net.pk, SerpAPI, HackerTarget, crt.sh, ip-api.com, proxycheck.io, public DNSBL operators) are operated independently of this application; their availability, accuracy, rate limits, and terms of service are outside this app's control and may change at any time.